



Mr. Unity Buddy

Posted on Aug 1, 2021 • Edited on Sep 20, 2021



61



5

C++ Cheat Sheet for Beginners 🔥

[#cpp](#) [#cheatsheet](#) [#beginners](#)

Hello, buddies! I recently discovered the power of cheat sheets and thought of making a cheat sheet for C++, which has a myth as "too hard." 🙄 Any language, even Python can be hard if you didn't try.

C++, an extension of C—which we said was an easy language to learn—is a general-purpose programming language. Google Chromium browser, several Microsoft applications, and even financial modeling at Morgan Stanley are said to be done with C++.

If you say it is hard, it is only for 3 reasons.

- It has complex syntax to support versatility
- It is a permissive language—you can do everything that's technically possible, even if not logically right

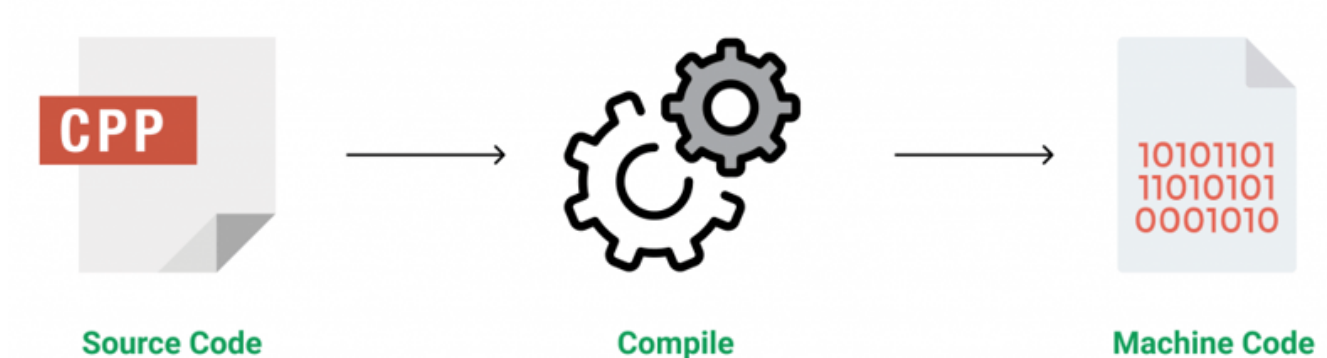
- It is best learned by someone who already has a foundation with C programming

Okay, these are just to note for a while. Now forget all about the things, take a deep breathe and start! This won't be boring, I will share games with you, so you can relax! 😊

Introduction

What is C++?

C++ is a general-purpose programming language that was developed as an enhancement of the C language to include object-oriented paradigm. It is an imperative and a compiled language.



C++ is a middle-level language rendering it the advantage of programming low-level (drivers, kernels) and even higher-level applications (games, GUI, desktop apps etc.). The basic syntax and code structure of both C and C++ are the same.

How C++ is Used?

C++ is used for all kinds of applications from computer games to OS and compilers. This programming language adds onto C Programming language

and includes modern programming

C++ is so compatible with C that it can compile more than 99% of the C programs without changing even a single source code

Anything that computers can physically perform from manipulating numbers to text can be programmed using C++.

SYNTAX

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    return 0;
}
```

Let's break up the code to understand it better:

- **Line 1** - `#include <iostream>` is a **header file library** that lets us work with input and output objects, such as `cout` (used in line 5). Header files add functionality to C++ programs.
- **Line 2**- `using namespace std` means that we can use names for objects and variables from the standard library.

Don't worry about these. Just think of it as something that (almost) always appears in your program.

- **Line 3** - Blank line. C++ don't care about spaces.
- **Line 4** - Another thing that always appear in a C++ program, is `int main()`.

This is called a function. Any code inside its curly brackets `{}` will be executed.

- **Line 5**- `cout` (Pronunciation - "see out") is an **object** used together with the insertion operator (`<<`) to output/print text. In our example it will output "Hello World". It's easy to remember this from pronunciation "see out".
- **Line 6** - `return 0` ends the main function.

⚠ *Things to note:*

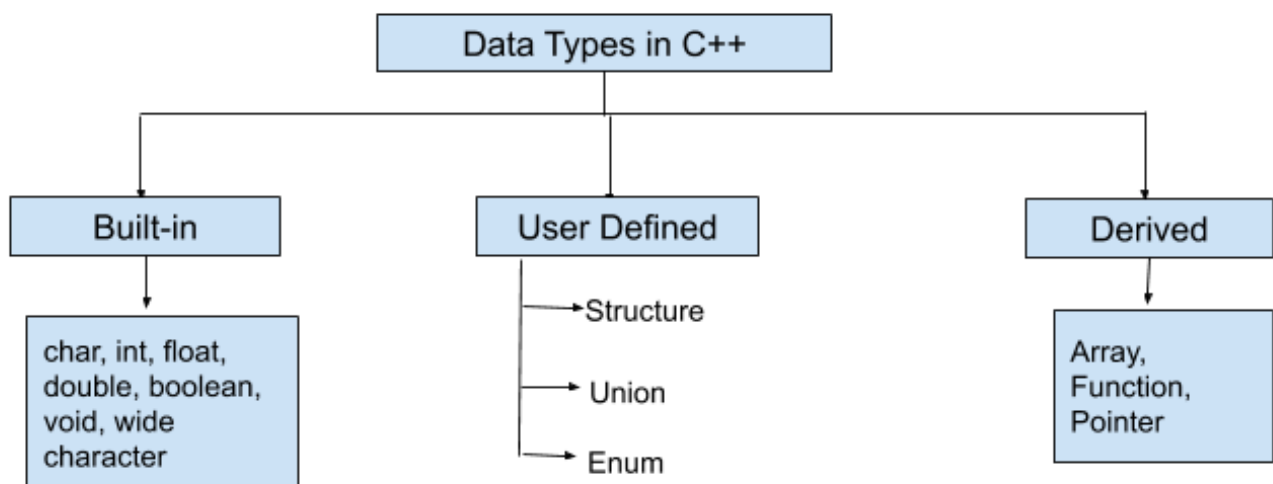
- Every line in C++ is ending with semicolon `;`.
- Do not forget to add the closing curly bracket `}` to actually end the main function.

COMMENTS

- Comments in C++ can be single or multi-line and start with `/*` and close with `*/`
- Comments can also start with `//`, extending to the lines end.

DATA TYPES

Type	Size(byte)	Symbol	Range
char/signed char	1	%c	-128 to 127
unsigned char	1	%c	0 to 255
Short int /signed short int	1	%d	-128 to 127
unsigned int	1	%u	0 to 255
int /signed int	2	%d	-32768 to 32767
unsigned int	2	%u	0 to 65535
long int/signed long int	4	%ld	-2147483648 to 214748647
unsigned long int	4	%lu	0 to 4294967295
float	4	%f	-3.4e38 to 3.4e38
double	8	%lf	-3.4e308 to 3.4e308
long double	10	%lf	-1.7e4982 to 1.7e4982



Let's learn more about Data types in Future. 😊

VARIABLES

Variables are containers for storing data values.

In C++, there are different types of variables (defined with different keywords), for example:

- `int` - stores integers (whole numbers), without decimals, such as 123 or

-123

- `double` - stores floating point numbers, with decimals, such as 19.99 or -19.99
- `char` - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- `string` - stores text, such as "Hello World". String values are surrounded by double quotes
- `float` - stores Boolean values with two states: true or false

A variable in C++ must be a specified data type

```
int myNum = 12; // Integer (whole number without decimals)
string name = "Unity Buddy"; // String(text)
char myLetter = 'U'; //character
float myFloatNum = 5.99; // Floating point number (with decimals)
```

USER INPUT

We have already learnt about `cout` which is used for Show Output. So for take **user input**, we use opposite side, `cin`, it means "See In". **See out** and **See In**, It's easy! Let's take an example.

```
int x; // making a variable called x.
cout << "Type a number: "; // Type a number and press enter
cin >> x; // Get user input from the keyboard
cout << "Your number is: " << x; // Display the input value
```

⚠ Remember,

- `cout` is used to show Output and uses the insertion operator (`<<`)
- `cin` is used to get User Input and uses the extraction operator (`>>`)

STRINGS

- A `string` variable contains a collection of characters surrounded by double quotes: `""c++`

```
string myName = "Mr Unity Buddy"
```

```
To use strings, you must import <string> library.  
""c++
```

```
#include <string>
```

String Concatenation

To combine two strings, we use `+` sign.

```
string name = "Unity Buddy";  
string age = "1102 years old."  
cout << name + age;
```

String Length

To find the length of the string, we can use `length()` function. (`size()`)

function is also used for the same thing.)

```
string name = "Unity Buddy"
cout << "The length of your name is:" << name.length()
```

IF... ELSE

C++ supports the usual logical conditions from mathematics:

- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`
- Equal to: `a==b`
- Not Equal to: `a != b`

If

In C++, we use the if statement to specify a block of C++ code to be executed if a condition is true.

```
if (20 > 18) {
    cout << "20 is greater than 18";
}
```


Else

The `else` statement is to specify a block of code to be executed if the condition is `false`.

```
int time = 20;
if (time < 18) {
    cout << "Good day.";
} else {
    cout << "Good evening.";
}
// Outputs "Good evening."
```

Else if

Use the `else if` statement to **specify a new condition** if the first condition is false.

```
if (condition1) {
    // block of code to be executed if condition1 is true
} else if (condition2) {
    // block of code to be executed if the condition1 is false and condition2 is true
} else {
    // block of code to be executed if the condition1 is false and condition2 is false
}
```

LOOPS

Loops can execute a block of code as long as a specified condition is reached. Loops are handy because they save time, reduce errors, and they make code more readable.

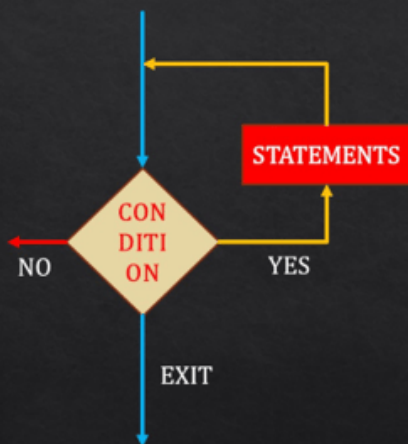
While Loop

The while loop loops through a block of code **as long as a specified condition is true**:

```
int main() {
    int i = 0;
    while (i < 5) {
        cout << i << "\n" ; //make new lines for each
        i++; //Add +1.
    }
    return 0;
/* output - 1
            2
            3
            4
            5
*/
}
```

The Do/While Loop

WHILE, FOR LOOPS



DO WHILE LOOP



The do/while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

```
int i = 0;
do {
    cout << i << "\n";
    i++;
}
while (i < 5);
```

It means, While you're reading this, your knowledge improves (+1) Hope you got it.

For Loop

When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop. Think you get \$50 for hour if you work till 12.30pm and you always working till 12.30pm. You are sure about

it. So you have to use a `for` loop to estimate your payment.

```
for (bool works = true; works = true; pay = pay+50 ) {  
    cout << pay << "\n";  
}  
//this is not the best way to do it. Just to understand
```

If you're not sure you work till 12.30pm, you have to use a while loop.

```
while (bool works = true) {  
    cout << pay << "\n";  
    pay = pay+50;  
}
```

Break and Continue

The `break` statement can also be used to jump out of a loop.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    cout << i << "\n";  
}
```

And the `continue` statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of 4:

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    cout << i << "\n";  
}
```

That's about loops!

ARRAYS

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

To declare an array, define the variable type, specify the name of the array followed by square brackets and specify the number of elements it should store:

```
string games[4];
```

We have now declared a variable that holds an array of four strings. To insert values to it, we can use an array literal - place the values in a comma-separated list, inside curly braces:

```
string games[4] = {"COD", "Cyberpunk", "Among Us", "Warplanes"} ;
```

Access the Elements of an Array

You access an array element by referring to the index number.

Note - Array indexes start with 0; [0] is the first element. [1] is the second element, etc.

```
string games[4] = {"COD", "Cyberpunk", "Among Us", "Warplanes"} ;  
cout << games[0];  
//Outputs COD
```

To change an element in an array,

```
string games[4] = {"COD", "Cyberpunk", "Among Us", "Warplanes"} ;  
games[0] = "Overwatch";  
cout << games[0]  
// Outputs Overwatch
```

FUNCTIONS

Last Part! You're a awesome learner!

C++ provides some pre-defined functions, such as `main()`, which is used to execute code. But you can also create your own functions to perform certain actions.

To create (often referred to as declare) a function, specify the name of the function, followed by parentheses `()`:

```
void buddyFunction(){  
  // code to be executed  
}
```

Call a Function

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are called.

To call a function, write the function's name followed by two parentheses `()` and a semicolon;

In the following example, `buddyFunction()` is used to print a text (the action), when it is called:

```
void buddyFunction(){  
  cout << "I am happy now!"  
  
int main() {  
  myFunction(); // call the function  
  return 0;  
}  
//Outputs "I am happy now!"
```

So buddies, this is the end and hope you learnt something. Did I miss something that must be included? Please let me know in comments!

And, Thanks for reading this long cheat sheet and Happy Coding!

[Originally](#) published on Hashnode

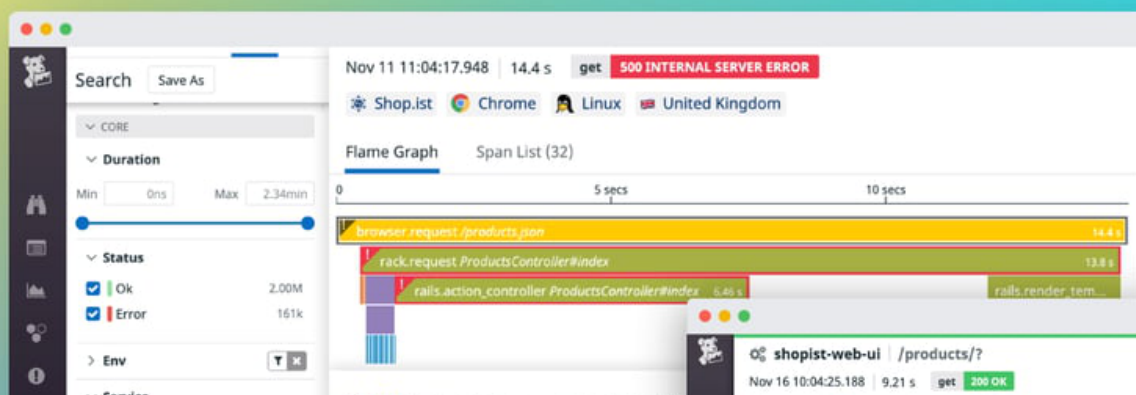


Datadog

PROMOTED



Quick Guide on Front-End Development



[Read More](#)

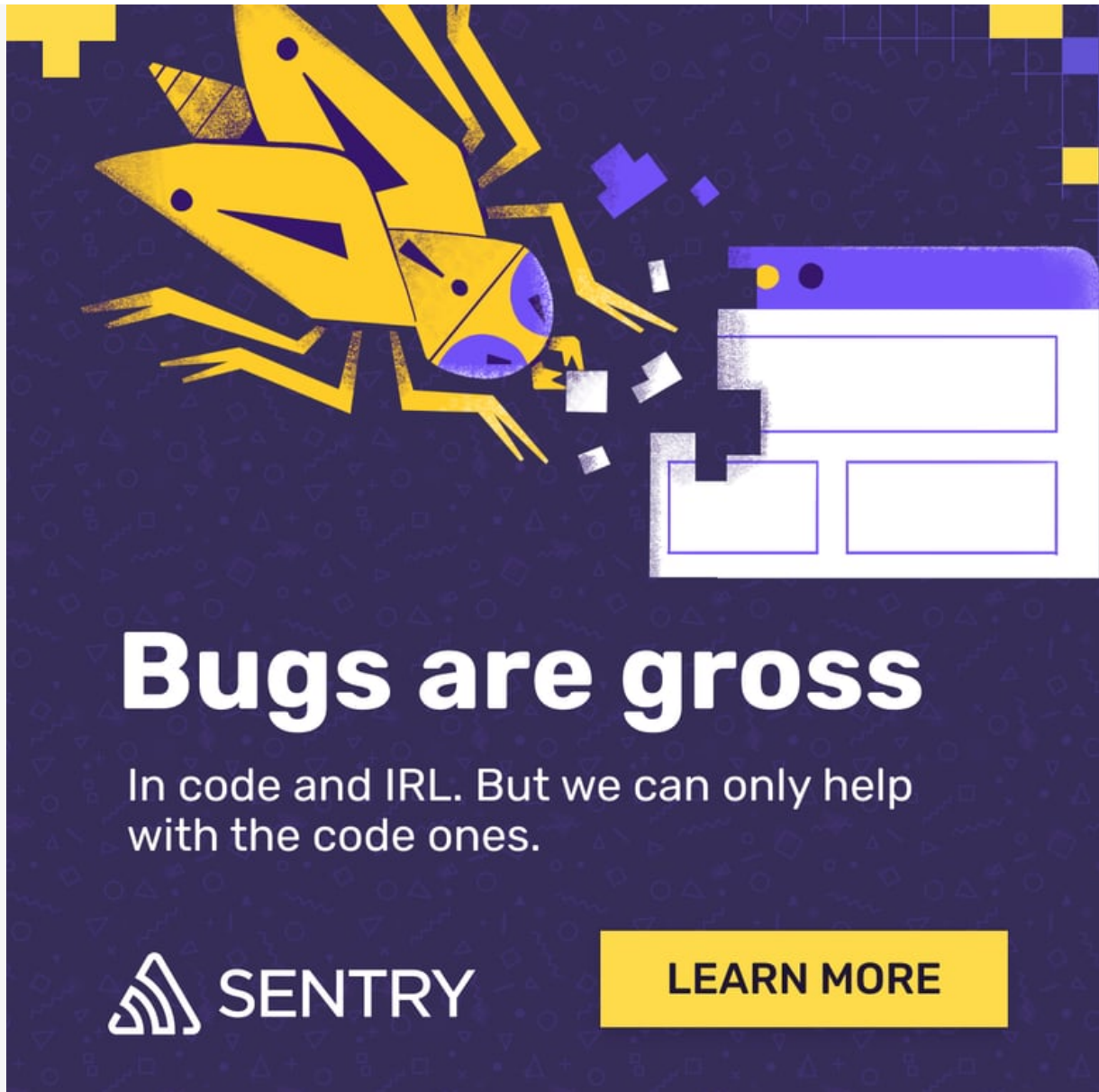
Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



Sentry

PROMOTED

The advertisement features a dark blue background with a pattern of small, light blue geometric shapes. In the upper left, a large, stylized yellow bug is depicted, appearing to be crawling or falling towards the right. To its right, a white rectangular shape, resembling a code editor or a form, is shown with a yellow bug icon on its top bar. Below the bug and the white shape, the text "Bugs are gross" is written in a large, bold, white sans-serif font. Underneath this, in a smaller white font, is the text "In code and IRL. But we can only help with the code ones." At the bottom left, the Sentry logo (a stylized 'S' made of three curved lines) is followed by the word "SENTRY" in a bold, white, sans-serif font. At the bottom right, there is a yellow rectangular button with the text "LEARN MORE" in a bold, black, sans-serif font.

Bugs are gross

In code and IRL. But we can only help with the code ones.

 SENTRY

LEARN MORE

[See why 4M developers consider Sentry, "not bad."](#)

Fixing code doesn't have to be the worst part of your day. Learn how Sentry can help.

[Learn more](#)



Mr. Unity Buddy

Technical Writer • Content Creator • Game Designer • Mad Developer

WORK

Content Creator at JustT Studios

JOINED

Mar 7, 2021

More from [Mr. Unity Buddy](#)

What To Do If Coding Is Hard..?

[#beginners](#) [#programming](#) [#tutorial](#) [#productivity](#)

5 Ways To Stay Motivated As A Developer

[#beginners](#) [#programming](#) [#productivity](#) [#career](#)

Good Code, Bad Code: Start To Think Like A Software Engineer

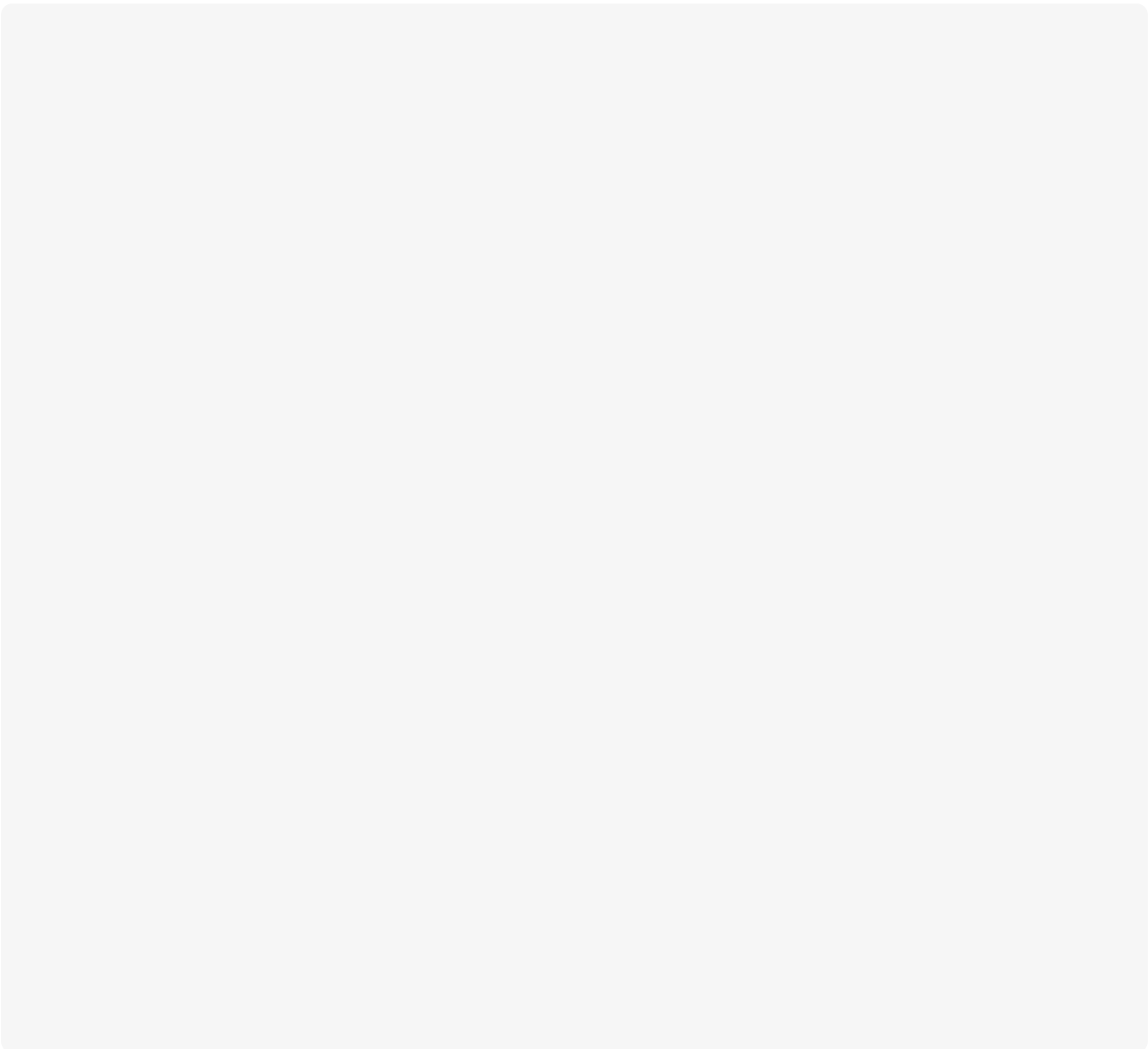
[#programming](#) [#beginners](#) [#productivity](#) [#career](#)



Heroku

PROMOTED





Simplify your DevOps and maximize your time.

Since 2007, Heroku has been the go-to platform for developers as it monitors uptime, performance, and infrastructure concerns, allowing you to focus on writing code.

[Learn More](#)

